

Part 01:

GitHub: <https://github.com/janaashraf888/rowadmasrtasks.git>

Question:

Why is it recommended to explicitly assign values to enum members in some cases?

Answer:

It is recommended to explicitly assign values to enum members when you need precise control over their numeric representation. This is especially important when working with databases, external APIs, file storage, or legacy systems where specific numeric values must match predefined constants. Explicit assignment also improves code clarity and prevents unexpected value shifts if new members are inserted between existing ones.

Question:

What happens if you assign a value to an enum member that exceeds the underlying type's range?

Answer:

If you assign a value that exceeds the underlying type's range, a compile-time error will occur. Enums are limited by the range of their underlying type (such as byte, short, or int). If the value does not fit within that range, the compiler prevents the assignment to maintain type safety.

Question:

What is the purpose of the virtual keyword when used with properties?

Answer:

The virtual keyword allows a property to be overridden in a derived class. This enables polymorphism, meaning the derived class can provide a different implementation of the property while still maintaining the same interface defined in the base class.

Question:

Why can't you override a sealed property or method?

Answer:

A sealed property or method cannot be overridden because the sealed keyword prevents further modification in derived classes. It is used to stop inheritance at that level and ensure that the implementation remains unchanged for all subclasses.

Question:

What is the key difference between static and object members?

Answer:

Static members belong to the class itself and are shared among all instances. They can be accessed without creating an object. Object (instance) members belong to a specific instance of a class and require object creation before they can be accessed.

Question:

Can you overload all operators in C#? Explain why or why not.

Answer:

No, not all operators can be overloaded in C#. Only specific operators defined by the language can be overloaded. Operators such as =, &&, ||, and ?: cannot be overloaded because their behavior is fundamental to the language's internal logic and execution rules.

Question:

When should you consider changing the underlying type of an enum?

Answer:

You should consider changing the underlying type of an enum when memory optimization is important or when working with external systems that require a specific numeric type. For example, using byte instead of int can reduce memory usage when storing large collections of enum values.

Question:

Why can't a static class have instance constructors?

Answer:

A static class cannot have instance constructors because it cannot be instantiated. Static classes are designed to contain only static members, and creating an object of a static class is not allowed in C#.

Question:

What are the advantages of using Enum.TryParse over direct parsing with int.Parse?

Answer:

Enum.TryParse is safer because it does not throw exceptions if the input is invalid. Instead, it returns a boolean value indicating whether parsing succeeded. This allows graceful error handling and improves application stability compared to int.Parse, which throws exceptions on invalid input.

Question:

What is the difference between overriding Equals and == for object comparison in C# struct and class?

Answer:

Overriding Equals allows you to define custom logic for value comparison. In classes, the == operator by default compares references, while Equals can be overridden to compare actual object data. In structs, Equals by default performs value comparison, but == must be explicitly overloaded if custom comparison behavior is needed.

Question:

Why is overriding ToString beneficial when working with custom classes?

Answer:

Overriding ToString provides a meaningful string representation of an object. This improves debugging, logging, and output readability by displaying useful information instead of the default type name.

Question:

Can generics be constrained to specific types in C#? Provide an example.

Answer:

Yes, generics can be constrained using the where keyword. Constraints ensure that the generic type meets certain requirements.

Example:

```
public T Max<T>(T a, T b) where T : IComparable
```

This constraint ensures that T implements the IComparable interface.

Question:

What are the key differences between generic methods and generic classes?

Answer:

A generic method defines type parameters only for that specific method, while a generic class defines type parameters for the entire class. Generic classes maintain type consistency across all their

members, whereas generic methods provide flexibility within a single method.

Question:

Why might using a generic swap method be preferable to implementing custom methods for each type?

Answer:

A generic swap method promotes code reusability and reduces duplication. Instead of writing separate swap methods for int, double, string, etc., a single generic method can handle all types while maintaining type safety.

Question:

How can overriding Equals for the Department class improve the accuracy of searches?

Answer:

Overriding Equals ensures that two Department objects are compared based on their actual data (such as Id and Name) rather than their memory references. This improves the accuracy of search operations when comparing objects in arrays or collections.

Question:

Why is == not implemented by default for structs?

Answer:

The == operator is not automatically implemented for structs

because C# requires explicit definition to ensure the developer intentionally defines comparison logic. While structs compare by value using Equals by default, == must be manually overloaded to avoid ambiguity in how equality should be determined.

Part 02:

Question:

What do we mean by the Generalization concept using Generics?

Answer:

Generalization using Generics means writing code in a way that works with different data types without being restricted to a specific one. Instead of creating separate classes or methods for int, string, double, etc., we define a generic type parameter (such as T) that represents any type.

This approach promotes code reusability, type safety, and flexibility.

The compiler ensures that the correct type is used, reducing runtime errors and eliminating the need for type casting.

For example, instead of writing separate methods like:

- SwapInt(int a, int b)
- SwapDouble(double a, double b)

We can write a single generic method:

- Swap<T>(T a, T b)

This is called generalization because the solution is generalized to work with multiple types.

Question:

What do we mean by hierarchy design in real business?

Answer:

Hierarchy design in real business refers to organizing classes in a structured parent–child relationship that reflects real-world business models. It uses inheritance to represent common features in a base class and specialized features in derived classes.

For example, in a company system:

- Employee (base class)
- Manager, Developer, Accountant (derived classes)

All employees share common properties like Name and Salary, while each role may have additional specific behaviors.

Hierarchy design improves:

- Code organization
 - Reusability
 - Maintainability
- Real-world modeling accuracy

LinkedIn Post:

https://www.linkedin.com/posts/janaashraf88_csharp-dotnet-oop-activity-7430424394147127296-Rgp9?utm_source=share&utm_medium=member_desktop&rcm=ACoAAE8YvOUBdlbw_Me3Yv_nmlqA8meTZp5UEhc

Part03 Bonus:

1- Couple Operators

Couple operators are operators used together to perform a specific action or compare two values in combination.

- **Arithmetic coupling:** +=, -=, *=, /= → combines arithmetic and assignment.
- **Comparison coupling:** ==, !=, >, <, >=, <= → used to evaluate conditions.

Purpose: Helps in writing **concise, readable code** by combining operations.

2- const vs readonly

Both **const** and **readonly** declare values that cannot change, but they differ in how and when the value is set:

- **Initialization:**
 - **const** → must be initialized at **compile time**.
 - **readonly** → can be initialized at **runtime**, either in declaration or in the constructor.
- **Scope:**
 - **const** → always **static** by default.
 - **readonly** → can be **instance-level** or **static**.
- **Modification:**
 - **const** → cannot be changed anywhere.

- readonly → can only be assigned in the **constructor**.
 - **Use case:**
 - const → fixed values like PI or MaxValue.
 - readonly → values fixed per instance or determined at runtime.
-

3- Partial Class – Object

A **partial class** allows a class definition to be **split across multiple files**.

- Helps in **team development** and **code organization**.
- Separates **auto-generated code** from **custom code**.
- After compilation, all partial parts are **combined into a single class object**.

Example usage:

- File1 → class properties
 - File2 → class methods
-

4- Singleton Design Pattern

The **Singleton Pattern** ensures a class has **only one instance** and provides a **global point of access**.

- **Key points:**
 - Private constructor → prevents external instantiation.

- Static property → stores the single instance.
 - Lazy initialization → creates the instance only when needed.
 - **Use cases:** Logging, configuration, database connections.
-

5- readonly Keyword

The **readonly** keyword declares fields whose value can only be assigned:

- During **declaration**, or
- In the **constructor**

Purpose: Provides **immutability at runtime** for instance-specific values.

Difference from const:

- readonly → can be set at runtime.
- const → must be set at compile time.

Example usage:

An Employee class can have a **readonly Id** that is assigned in the constructor and cannot be changed afterwards.

2- Event-Driven Programming

Event-Driven Programming is a programming paradigm in which the **flow of the program is determined by events**.

- **Event:** Any action or occurrence that a program can respond to, such as:

- User actions (clicks, key presses, mouse movements)
- System-generated events (timers, messages, or signals)
 - External inputs (file changes, network messages)

- **Key concepts:**

- **Event Source:** The object that generates the event (e.g., a button).
- **Event Handler:** The method or function that responds to the event.
- **Event Listener/Subscriber:** The code that listens for the event and triggers the handler.

- **How it works:**

1. An event occurs (e.g., a user clicks a button).
2. The program detects the event through an **event listener**.
3. The **event handler** executes the appropriate response.

- **Benefits:**

- Makes programs **interactive** and **responsive**.
- Separates the **event detection** from **event handling**, improving code organization.
- Commonly used in **GUI applications, games, and real-time systems**.

Example in C#:

- A button click triggers a method to show a message or perform an action.

- The button is the **event source**, and the method is the **event handler**.